

Data Structures & Algorithms

Definitions, Python implementations, and complexity – for interview prep.

CONTENTS

What's inside

I	Arrays & Lists	VIII	Graphs
II	Strings	IX	Searching
III	Linked Lists	X	Graph Traversal
IV	Stacks & Queues	XI	Sorting
V	Hash Maps & Sets	XII	Recursion & Backtracking
VI	Trees	XIII	Dynamic Programming
VII	Heaps	XIV	Greedy Algorithms

ORIENTATION

How to read this deck

DEFINITION	A short, textbook-style statement of what the structure or technique is.
PYTHON	A tight, idiomatic implementation – the canonical shape, not the most optimized version.
COMPLEXITY	Time and space costs for each operation, expressed in Big-O.
WHEN TO USE	Brief guidance on the kinds of problems where this structure or technique is the right reach.

Big-O notation

An upper bound on running time as a function of input size n , ignoring constants and lower-order terms.

We care about the *shape* of the growth curve, not its exact height. Two linear algorithms belong to the same complexity class even if one is twice as fast — what matters is that doubling the input roughly doubles the work.

$O(2n + 5) = O(n)$. $O(n^2 + n) = O(n^2)$. $O(\log n) < O(n) < O(n \log n) < O(n^2)$.

PRELIMINARIES

Common complexities

$O(1)$ Constant

Hash map lookup · array indexing · stack push/pop

$O(\log n)$ Logarithmic

Binary search · balanced tree operations · heap push/pop

$O(n)$ Linear

Single pass over an array · linked list traversal

$O(n \log n)$ Linearithmic

Merge sort · quicksort (avg) · heapsort · any comparison sort

$O(n^2)$ Quadratic

Nested loops · bubble / insertion / selection sort

$O(2^n)$ Exponential

Recursive subset / power-set generation without memoization

SECTION ONE

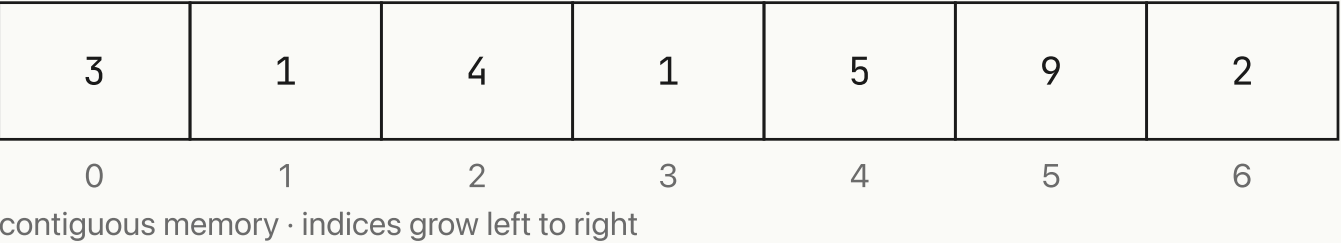
I

Arrays & Lists

Contiguous, indexable sequences. The structure underneath almost everything else.

Arrays / Lists

A contiguous block of memory holding elements accessible by integer index. Python's `list` is a dynamic array that resizes as it grows.



COMPLEXITY — LIST

OPERATION	AVG	NOTES
<code>a[i]</code>	<code>O(1)</code>	<i>Indexed access</i>
<code>a.append(x)</code>	<code>O(1)*</code>	<i>Amortized</i>
<code>a.insert(i,x)</code>	<code>O(n)</code>	<i>Shifts tail</i>
<code>a.pop()</code>	<code>O(1)</code>	<i>From the end</i>
<code>a.pop(0)</code>	<code>O(n)</code>	<i>Shifts left</i>
<code>x in a</code>	<code>O(n)</code>	<i>Linear scan</i>

List operations in Python

```
a = [3, 1, 4, 1, 5, 9, 2]
```

<code>a[0]</code>	<code># → 3</code>	<i>indexed access, $O(1)$</i>
<code>a[-1]</code>	<code># → 2</code>	<i>negative index from end</i>
<code>a[2:5]</code>	<code># → [4, 1, 5]</code>	<i>slice, returns new list</i>
<code>a.append(6)</code>	<code># → [..., 2, 6]</code>	<i>amortized $O(1)$</i>
<code>a.insert(0, 0)</code>	<code># → [0, 3, ...]</code>	<i>$O(n)$, shifts right</i>
<code>a.pop()</code>	<code># → 6</code>	<i>removes from the end, $O(1)$</i>
<code>len(a)</code>	<code># → 7</code>	<i>$O(1)$</i>
<code>sorted(a)</code>	<code># → new sorted list,</code>	<i>$O(n \log n)$</i>

list comprehension – the Pythonic derived list

```
squares = [x * x for x in a if x > 0]
```

Two pointers

Walk two indices through the array under some rule – usually closing in from the ends, or moving in lockstep. Replaces a nested loop with a linear sweep.

```
def two_sum_sorted(a, target):  
    # a is sorted ascending; find indices i < j with a[i] + a[j] = target.  
    i, j = 0, len(a) - 1  
    while i < j:  
        s = a[i] + a[j]  
        if s == target:  
            return (i, j)  
        elif s < target:  
            i += 1      # need a bigger sum → move left up  
        else:  
            j -= 1      # need a smaller sum → move right down  
    return None
```

Time: $O(n)$ · Space: $O(1)$

Sliding window

Maintain a contiguous range. Expand on one side, contract on the other, to keep an invariant. Each index is touched at most twice.

```
def longest_unique_substring(s):  
    seen = {}                # char → last index seen  
    left = best = 0  
    for right, ch in enumerate(s):  
        if ch in seen and seen[ch] ≥ left:  
            left = seen[ch] + 1    # shrink from the left  
        seen[ch] = right  
        best = max(best, right - left + 1)  
    return best
```

Time: $O(n)$ · Space: $O(k)$ for alphabet size k

SECTION TWO

II

Strings

Immutable sequences of characters. Every "modification" is a fresh allocation.

Strings

An immutable sequence of characters. Operations that look like mutation actually produce a new string – build with `"".join(parts)`, not `+=` in a loop.

COMPLEXITY — STRING

OPERATION	TIME	NOTES
<code>s[i]</code>	$O(1)$	<i>Indexed access</i>
<code>s + t</code>	$O(n + m)$	<i>Allocates a new string</i>
<code>s in t</code>	$O(n \cdot m)$	<i>Substring search, worst case</i>
<code>s.split(...)</code>	$O(n)$	<i>Linear in length</i>
<code>"".join(parts)</code>	$O(n)$	<i>Total characters in parts</i>
<code>s += t</code> (loop)	$O(n^2)$	<i>⚠ classic trap – use a list + join</i>

```
# The trap
out = ""
for piece in parts:
    out += piece      #  $O(n^2)$  total!

# The fix
out = "".join(parts) #  $O(n)$  total
```

String operations in Python

```
s = "the quick brown fox"
```

```
s[4]                # → 'q'
```

```
s[4:9]             # → 'quick'
```

```
s.split()          # → ['the', 'quick', 'brown', 'fox']
```

```
" ".join(["a", "b", "c"])    # → 'a b c'
```

```
"quick" in s        # → True    substring search
```

```
s.find("fox")       # → 16      index, or -1
```

```
s.replace("quick", "slow")   # → new string
```

```
s.upper()           # → 'THE QUICK BROWN FOX'
```

```
name, n = "Ada", 3
```

```
f"{name} × {n} = {name * n}"    # → 'Ada × 3 = AdaAdaAda'
```

SECTION THREE

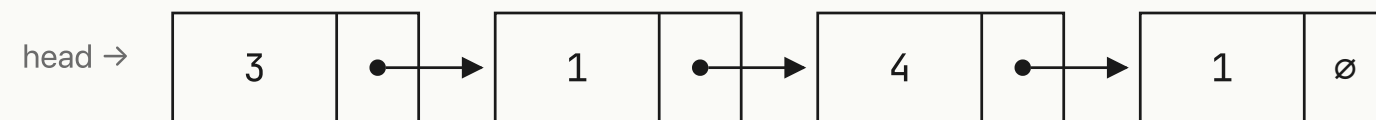
III

Linked Lists

A chain of nodes. Trade indexed access for constant-time insertion at any known node.

Singly linked list

Each node holds a value and a reference to the next node. The list itself is a reference to the head.



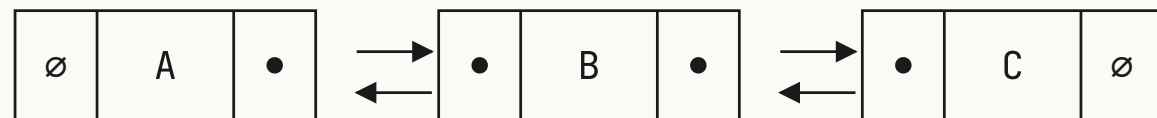
```
class Node:
    def __init__(self, value, nxt=None):
        self.value = value
        self.next = nxt

def prepend(head, value):
    return Node(value, head)           # O(1)

def traverse(head):
    node = head
    while node:
        yield node.value               # O(n) to walk
        node = node.next
```

Doubly linked list

Each node also stores a `prev` pointer. Bidirectional traversal and constant-time removal of a known node, at the cost of extra memory.



```
class DNode:
    def __init__(self, value):
        self.value = value
        self.prev = None
        self.next = None

def remove(node):
    # O(1) given the node
    if node.prev: node.prev.next = node.next
    if node.next: node.next.prev = node.prev

# Python's collections.deque is a doubly linked block-l
```

Reversing a linked list

```
def reverse(head):  
    prev, curr = None, head  
    while curr is not None:  
        nxt      = curr.next      # save the rest of the list  
        curr.next = prev          # flip current's pointer backwards  
        prev     = curr           # advance prev one step  
        curr     = nxt            # advance curr one step  
    return prev                  # new head
```

Time: $O(n)$ · Space: $O(1)$

Three pointers: prev, curr, next – no recursion needed.

SECTION FOUR

IV

Stacks & Queues

Disciplines of access, not layouts in memory. LIFO and FIFO, respectively.

Stacks

Last-in, first-out. The last item pushed is the first item popped. Natural fit for nested structure, function calls, undo, parenthesis matching.

9 ← top
5
1
3

→ push / pop

```
stack = []

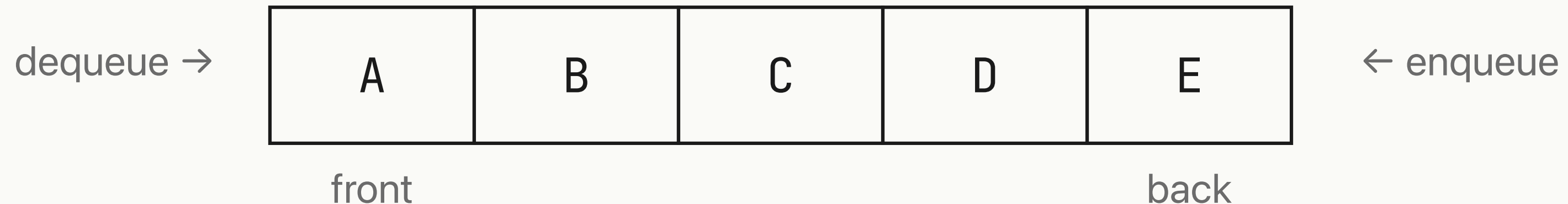
stack.append(3)           # push, 0(1)
stack.append(1)
stack.append(9)

stack[-1]                 # peek → 9, 0(1)
stack.pop()               # pop  → 9, 0(1)
```

Push, pop, peek are all $O(1)$ amortized.

Queues

First-in, first-out. Items enter the back and leave the front. Use `collections.deque` – `list.pop(0)` is $O(n)$.



```
from collections import deque

q = deque()
q.append("A")           # enqueue, O(1)
q.append("B")
front = q.popleft()     # dequeue → 'A', O(1)
```

Dequeues

A double-ended queue – push and pop at either end in constant time. A strict superset of both stack and queue.

```
from collections import deque

d = deque([1, 2, 3])

d.append(4)          # right push → deque([1, 2, 3, 4])
d.appendleft(0)      # left push  → deque([0, 1, 2, 3, 4])
d.pop()              # right pop  → 4
d.popleft()          # left pop   → 0

# All four operations are  $O(1)$ .
# Use a deque for: BFS frontiers, sliding-window extremes,
# rolling buffers, any structure that grows or shrinks at both ends.
```

Monotonic stack

A stack kept in sorted order by popping anything that would break the order before pushing. Each element is pushed and popped at most once → linear total.

```
def next_greater(a):  
    # For each index, the next greater value to its right (or -1).  
    result = [-1] * len(a)  
    stack = [] # indices, values decreasing  
    for i, x in enumerate(a):  
        while stack and a[stack[-1]] < x:  
            result[stack.pop()] = x  
        stack.append(i)  
    return result
```

```
# next_greater([2, 1, 3, 1, 4]) → [3, 3, 4, 4, -1]  
# Time: O(n) amortized · Space: O(n)
```

SECTION FIVE

V

Hash Maps & Sets

Average-case constant-time lookup. The single most useful structure in the kit.

Hash maps

Key-value pairs stored in an array indexed by $\text{hash}(\text{key})$. Average $O(1)$ for `get`, `set`, `del`. Python's `dict`.

```
counts = {}
for word in text.split():
    counts[word] = counts.get(word, 0) + 1

# Or, idiomatic:
from collections import Counter
counts = Counter(text.split())

# Common ops
counts["the"]           # lookup,  O(1) avg
counts["and"] = 5        # insert,  O(1) avg
del counts["the"]        # delete,  O(1) avg
"the" in counts          # member,  O(1) avg
for k, v in counts.items(): ... # iterate, O(n)
```

Hash sets

A hash map without values. Deduplication, membership tests, and set algebra in expected constant time per element.

```
seen = set()
unique = []
for x in items:
    if x not in seen:    # O(1) avg
        seen.add(x); unique.append(x)

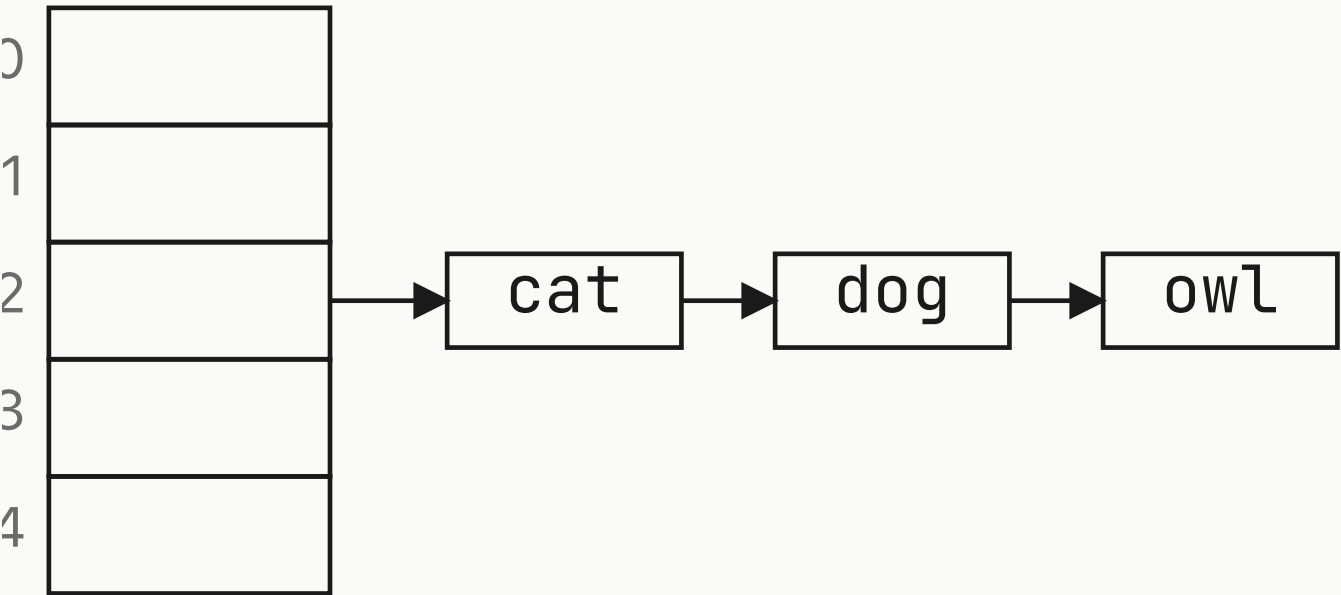
# Set algebra
a = {1, 2, 3, 4};    b = {3, 4, 5, 6}
a | b                # union          → {1, 2, 3, 4, 5, 6}
a & b                # intersection → {3, 4}
a - b                # difference   → {1, 2}

# Two-sum in O(n) with a map of complements:
def two_sum(a, t):
    seen = {}
    for i, v in enumerate(a):
        complement = t - v
        if complement in seen:
            return (i, seen[complement])
        seen[v] = i
```

Collision handling

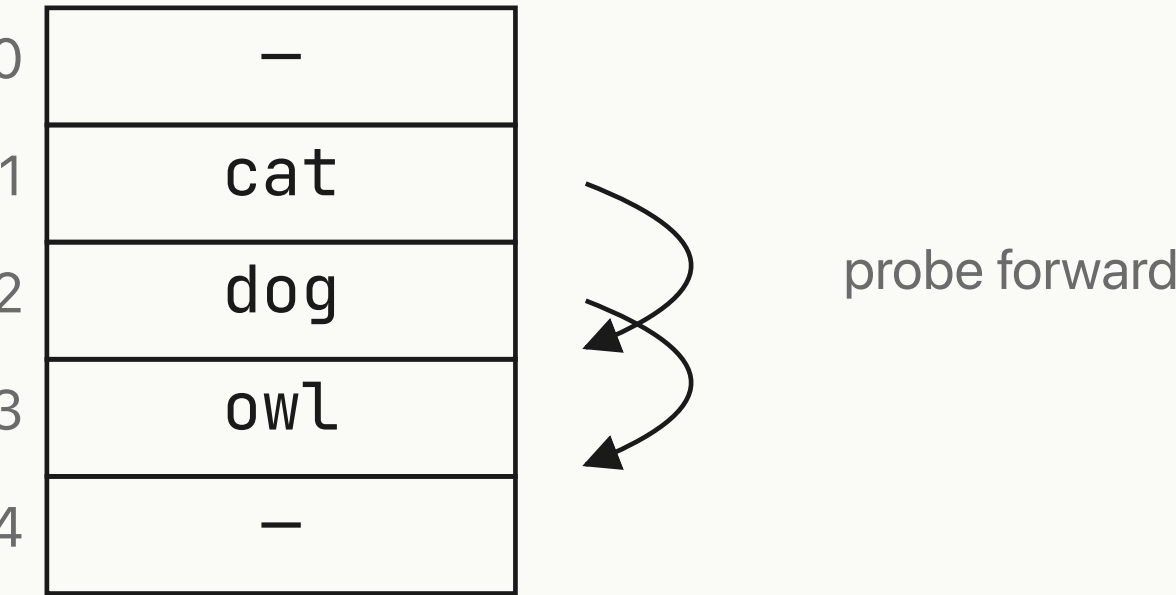
What happens when two keys hash to the same slot. Two strategies dominate.

CHAINING



Each slot holds a list. Lookup walks the chain.

OPEN ADDRESSING (PYTHON)



On collision, probe to the next empty slot.

SECTION SIX

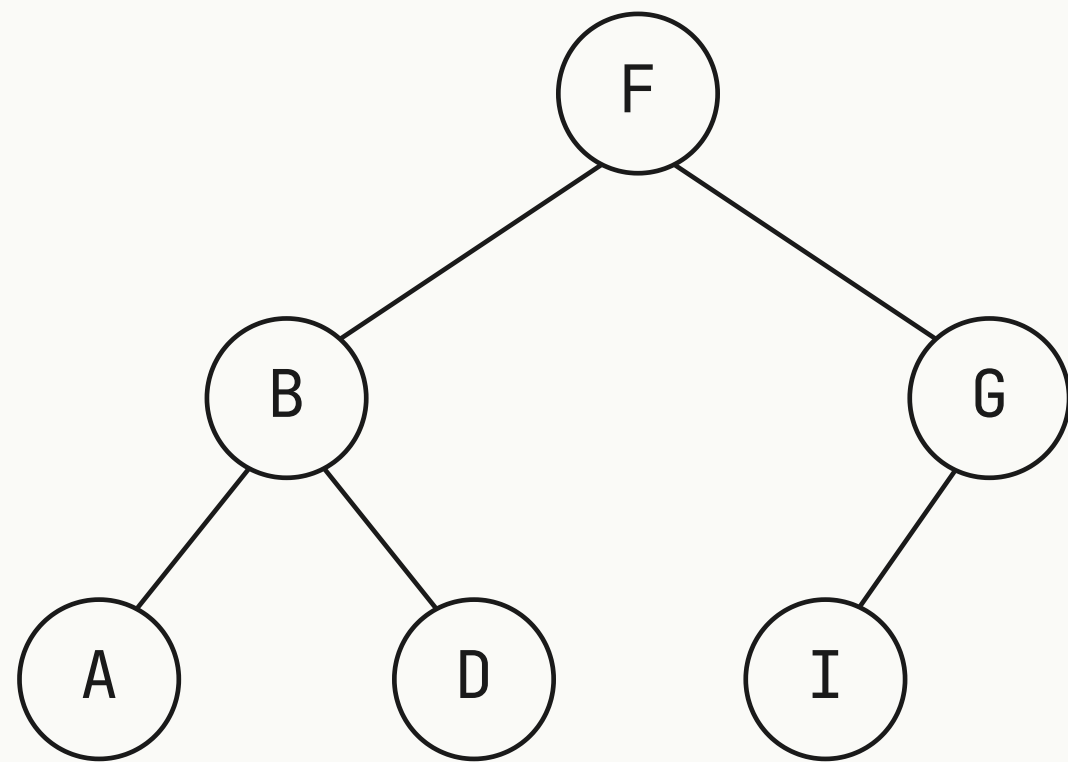
VI

Trees

Hierarchical structures. One root, each non-root with exactly one parent.

Binary trees

A tree where each node has at most two children – left and right. Height is $\log n$ if balanced, n if degenerate.



```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def height(node):
    if node is None:
        return 0
    return 1 + max(height(node.left),
                    height(node.right))
```

Tree traversals

Three depth-first orders, differing only in when the current node is visited. Plus one breadth-first.

```
def inorder(n):                # left, self, right    → sorted on a BST
    if n: inorder(n.left); yield n.value; yield from inorder(n.right)

def preorder(n):              # self, left, right    → tree copy / serialization
    if n: yield n.value; yield from preorder(n.left); yield from preorder(n.right)

def postorder(n):             # left, right, self    → bottom-up aggregation
    if n: yield from postorder(n.left); yield from postorder(n.right); yield n.value

def level_order(root):        # breadth-first with a queue
    q = deque([root])
    while q:
        n = q.popleft()
        if n:
            yield n.value
            q.append(n.left)
            q.append(n.right)
```

Binary search trees

A binary tree with an ordering invariant: left subtree < node < right subtree. Search, insert, delete in $O(\log n)$ if balanced – $O(n)$ if not.

```
def search(node, target):
    while node:
        if target == node.value: return node
        node = node.left if target < node.value else node.right
    return None

def insert(node, value):
    if node is None: return Node(value)
    if value < node.value: node.left = insert(node.left, value)
    else: node.right = insert(node.right, value)
    return node
```

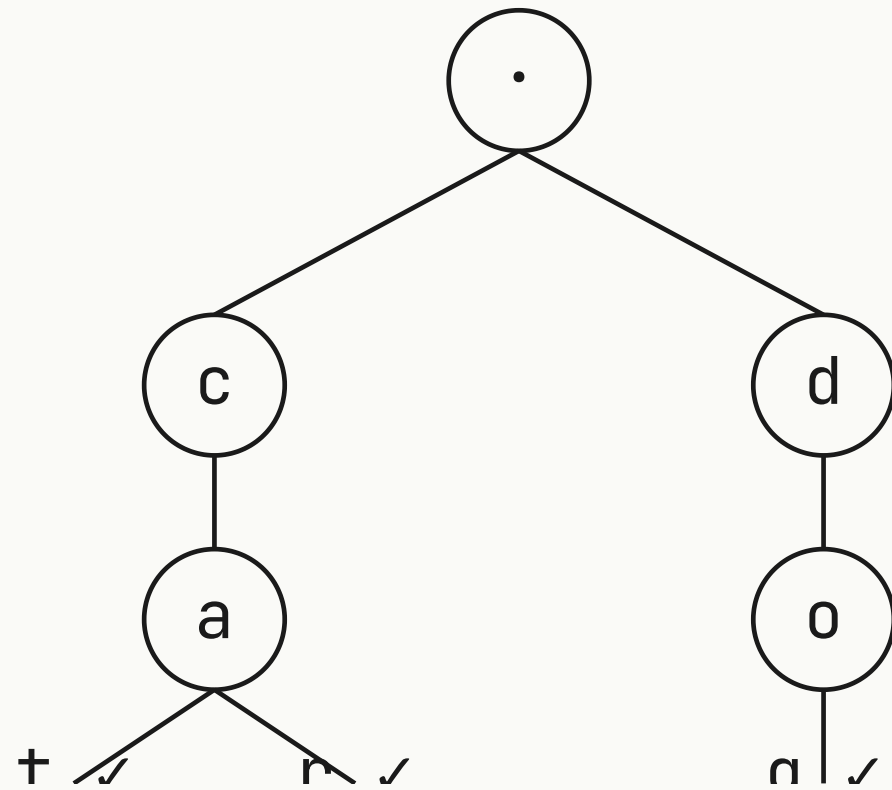
COMPLEXITY — BST

OPERATION	AVG	WORST
search	$O(\log n)$	$O(n)$
insert	$O(\log n)$	$O(n)$
delete	$O(\log n)$	$O(n)$

Self-balancing variants (red-black, AVL) restore the $O(\log n)$ worst case.

Tries

A tree where each edge is a character. Words sharing a prefix share a path. The right reach for autocomplete and prefix queries.



```
class Trie:
    def __init__(self):
        self.root = {}

    def insert(self, word):
        node = self.root
        for c in word:
            node = node.setdefault(c, {})
        node["$"] = True # end-of-word marker

    def starts_with(self, prefix):
        node = self.root
        for c in prefix:
            if c not in node: return False
```

SECTION SEVEN

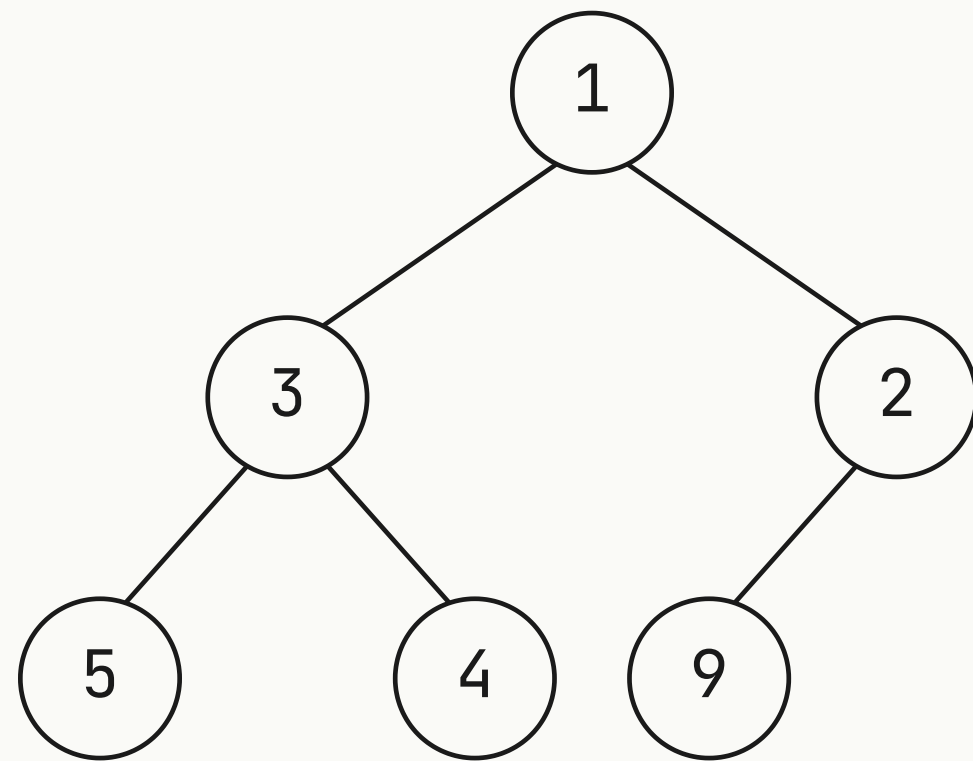
VII

Heaps

Tree-shaped, array-stored. Constant-time access to the extreme element.

Heaps / priority queues

A complete binary tree where every parent is smaller (min-heap) or larger (max-heap) than its children. Stored as a flat array – children of i live at $2i+1$ and $2i+2$.



Array form: [1, 3, 2, 5, 4, 9]

```
import heapq
```

```
h = []
```

```
heapq.heappush(h, 5)
```

```
heapq.heappush(h, 1)
```

```
heapq.heappush(h, 3)
```

```
h[0] # peek min → 1, O(1)
```

```
heapq.heappop(h) # pop min → 1, O(log n)
```

```
# Max-heap: push negated values.
```

```
heapq.heappush(h, -x)
```

```
-heapq.heappop(h)
```

Heapify & top-k

Building a heap from an arbitrary list takes $O(n)$ – not $O(n \log n)$ – because most of the work happens near the leaves.

```
import heapq

a = [9, 4, 7, 1, 3, 5, 2]
heapq.heapify(a)           # in-place,  $O(n)$ 

# k smallest –  $O(n \log k)$  using a max-heap of size k
def k_smallest(items, k):
    h = []
    for x in items:
        heapq.heappush(h, -x)
        if len(h) > k:
            heapq.heappop(h)
    return sorted(-x for x in h)

# Or the one-liners from heapq, which use the same idea internally:
```

SECTION EIGHT

VIII

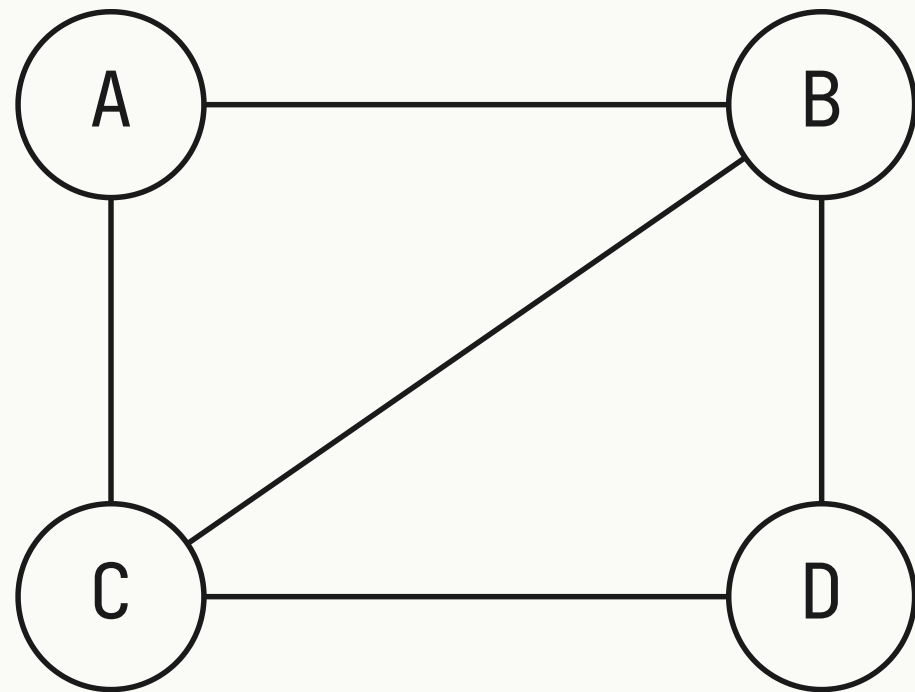
Graphs

Nodes and edges. The most general structure — models anything relational.

Graph representations

Two standard ways to put a graph in memory. Adjacency list is the default; the matrix is for dense graphs with frequent edge queries.

THE GRAPH



ADJACENCY LIST — $O(V + E)$ SPACE

```
graph = {  
  "A": ["B", "C"],  
  "B": ["A", "C", "D"],  
  "C": ["A", "B", "D"],  
  "D": ["B", "C"],
```

ADJACENCY MATRIX — $O(V^2)$ SPACE

	A	B	C	D
A	[0, 1, 1, 0]			
B	[1, 0, 1, 1]			
C	[1, 1, 0, 1]			

SECTION NINE

IX

Searching

Logarithmic time on sorted data – and on any monotonic search space.

Binary search

Repeatedly halve the search range. Requires a sorted (or otherwise monotonic) input.

```
def binary_search(a, target):  
    lo, hi = 0, len(a) - 1  
    while lo ≤ hi:  
        mid = (lo + hi) // 2  
        if a[mid] == target:  
            return mid  
        elif a[mid] < target:  
            lo = mid + 1  
        else:  
            hi = mid - 1  
    return -1
```

Time: $O(\log n)$ · Space: $O(1)$

Watch the off-by-ones: $lo \leq hi$, and $mid \pm 1$ on the updates.

Binary search, generalized

The same idea works on any monotonic predicate. `bisect` gives you the building blocks.

```
import bisect

a = [1, 2, 4, 4, 4, 7, 9]
bisect.bisect_left(a, 4)      # → 2    first index where 4 could go
bisect.bisect_right(a, 4)     # → 5    one past the last 4

# Binary search on the answer – "smallest x such that f(x) is True":
def first_true(lo, hi, f):
    while lo < hi:
        mid = (lo + hi) // 2
        if f(mid): hi = mid
        else:      lo = mid + 1
    return lo
```

SECTION TEN

X

Graph Traversal

From a node, visit everything reachable. Two strategies, both $O(V + E)$.

Breadth-first search

Explore level by level. Uses a queue. Finds the shortest path in an unweighted graph by number of edges.

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    q = deque([start])
    order = []
    while q:
        node = q.popleft()
        order.append(node)
        for nbr in graph[node]:
            if nbr not in visited:
                visited.add(nbr)
                q.append(nbr)
    return order
```

"Time: $O(V + E)$ Space: $O(V)$ "

Depth-first search

Dive as deep as possible before backtracking. Uses a stack (or recursion). Natural fit for cycles, topological sort, connected components.

```
def dfs(graph, start):  
    visited, order = set(), []  
    def walk(node):  
        if node in visited: return  
        visited.add(node); order.append(node)  
        for nbr in graph[node]:  
            walk(nbr)  
    walk(start)  
    return order
```

```
# Time:  $O(V + E)$  · Space:  $O(V)$   
# Iterative variant: swap the call stack for an explicit list  
# stack, then pop() and append() unvisited neighbors.
```

SECTION ELEVEN

XI

Sorting

The most studied problem in CS. Two comparison sorts you should know cold.

Merge sort

Divide and conquer. Split, recursively sort, then merge. Guaranteed $O(n \log n)$; stable; uses $O(n)$ extra space.

```
def merge_sort(a):
    if len(a) ≤ 1: return a
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return _merge(left, right)

def _merge(left, right):
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    out.extend(left[i:]); out.extend(right[j:])
```

Quick sort

Pick a pivot, partition, recurse. Average $O(n \log n)$ with small constants; worst case $O(n^2)$ on bad pivots – so randomize.

```
def quick_sort(a):  
    if len(a) ≤ 1:  
        return a  
    pivot = a[len(a) // 2]  
    less = [x for x in a if x < pivot]  
    equal = [x for x in a if x == pivot]  
    greater = [x for x in a if x > pivot]  
    return quick_sort(less) + equal + quick_sort(greater)
```

*# Time: $O(n \log n)$ avg, $O(n^2)$ worst · Space: $O(\log n)$ avg
In-place version partitions in place and recurses on the two halves.*

Sorting at a glance

ALGORITHM	BEST	AVERAGE	WORST	SPACE	STABLE
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	<i>yes</i>
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	<i>no</i>
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	<i>no</i>
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	<i>yes</i>
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	<i>yes</i>
Timsort (Python)	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	<i>yes</i>

Python's `sorted()` and `list.sort()` use Timsort – a hybrid that exploits pre-existing order in the data.

SECTION TWELVE

XII

Recursion & Backtracking

A function that calls itself on a smaller version of the same problem.

Recursion

Every recursive solution needs two pieces: a base case that terminates, and a recursive case that makes progress toward it.

```
def factorial(n):  
    # base case  
    if n ≤ 1:  
        return 1  
    # recursive case  
    return n * factorial(n - 1)
```

Mental model: trust the recursion.

Assume factorial(n - 1) gives the right answer for n - 1,

then write the line that combines it into the answer for n.

Backtracking

Recursion that explores choices. Try, recurse, undo. The shape behind permutations, subsets, n-queens, sudoku.

```
def permutations(items):
    result, n = [], len(items)
    used = [False] * n
    path = []

    def backtrack():
        if len(path) == n:
            result.append(path.copy()); return
        for i in range(n):
            if used[i]: continue
            used[i] = True; path.append(items[i])    # try
            backtrack()                             # recurse
            used[i] = False; path.pop()              # undo

    backtrack()
```

SECTION THIRTEEN

XIII

Dynamic Programming

Overlapping subproblems, optimal substructure. Cache the answers.

DP – memoization

Top-down. Write the natural recursion, then cache each subproblem's answer so it's computed once.

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n < 2: return n
    return fib(n - 1) + fib(n - 2)
```

Without the cache: $O(2^n)$ calls – every $\text{fib}(k)$ recomputed exponentially often.

With the cache: $O(n)$ calls – each subproblem solved exactly once.

Time: $O(n)$ · Space: $O(n)$ for the cache + recursion stack.

DP – tabulation

Bottom-up. Build an array of subproblem solutions, starting from the base case and filling forward.

```
def fib(n):  
    if n < 2:  
        return n  
    dp = [0] * (n + 1)  
    dp[1] = 1  
    for i in range(2, n + 1):  
        dp[i] = dp[i - 1] + dp[i - 2]  
    return dp[n]
```

Time: $O(n)$ · Space: $O(n)$

Space-optimized – we only ever need the last two values:

```
def fib(n):  
    a, b = 0, 1  
    for _ in range(n):
```

SECTION FOURTEEN

XIV

Greedy Algorithms

Locally optimal choices, hoping for a globally optimal answer.

Activity selection

Given activities with start and end times, pick the maximum non-overlapping subset.

Greedy rule: always take the activity that ends earliest.

```
def activity_selection(activities):  
    # activities: list of (start, end) tuples  
    activities = sorted(activities, key=lambda x: x[1])    # by end time  
    chosen, last_end = [], float("-inf")  
    for start, end in activities:  
        if start ≥ last_end:  
            chosen.append((start, end))  
            last_end = end  
    return chosen
```

```
# Time:  $O(n \log n)$  for the sort    ·    Space:  $O(1)$  extra  
# The greedy choice is provably optimal here – an exchange argument.  
# Greedy fails on 0/1 knapsack; use DP there.
```

Complexity cheat sheet

STRUCTURE	ACCESS	SEARCH	INSERT	DELETE	SPACE	NOTES
Array / List	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	<i>Append $O(1)$ amortized</i>
String	$O(1)$	$O(n \cdot m)$	$O(n)$	$O(n)$	$O(n)$	<i>Immutable – joins, not +=</i>
Linked List	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$	$O(n)$	<i>* at a known node</i>
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	<i>LIFO – push/pop at top</i>
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	<i>FIFO – use deque</i>
Deque	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	<i>Both ends</i>
Hash Map / Set	–	$O(1)$	$O(1)$	$O(1)$	$O(n)$	<i>Avg; worst $O(n)$</i>
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	<i>Unbalanced $\rightarrow O(n)$</i>
Trie	$O(k)$	$O(k)$	$O(k)$	$O(k)$	$O(\Sigma \cdot n \cdot k)$	<i>k = key length</i>
Heap	$O(1)$ peek	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$	<i>Build: $O(n)$</i>
Graph (adj list)	–	$O(V + E)$	$O(1)$	$O(\deg)$	$O(V + E)$	<i>BFS / DFS: $O(V + E)$</i>

END

Practice by writing.

Read these implementations once. Then close the deck and write them out from memory. That's where the learning actually happens.